

The Free LLM API Playbook

The 16 permanent free providers, the gateway that connects them to Claude Code, and the exact settings that keep a session alive when your limit hits.

Why I built this

I kept hitting the same wall at the worst moment. Half way through a build, the usage limit lands, and the next few hours are gone. The fix I landed on is not a better plan. It is a second tier of models sitting behind the first one, free, already authenticated, ready to pick up the same conversation.

The list everything here is built on is **Awesome Free LLM APIs**, a GitHub directory maintained by mnfst. It catalogues 16 providers that give away **permanent** free API access to text models, with the base URL, the model names and the rate limits for each. Trial credits and time-limited promos are deliberately excluded, which is the reason it holds up as something you can depend on rather than something you try once.

What the list does not include is the plumbing. Every endpoint on it is OpenAI SDK-compatible, and Claude Code speaks the Anthropic Messages format, so there is a translation step in the middle that nobody mentions in the comments. This guide is that step, written out, with the two environment variables, the test that proves the path works, and the errors you will actually see.

01

How it actually works

1. You collect free API keys from the providers on the list. Google Gemini, Groq and NVIDIA NIM are the three I would start with, because all three are a signup with no credit card and all three carry daily request ceilings in the thousands rather than monthly call counts in the hundreds.
2. An LLM gateway sits between Claude Code and those providers. It accepts a request at POST `/v1/messages` in the Anthropic format, rewrites it into an OpenAI-format chat completion, calls whichever free provider is next in the chain, and converts the answer back on the way out. Your provider keys live on the gateway, not on your machine.
3. When a provider returns a rate-limit error, the gateway forwards the identical message array to the next model in the chain. Context carries because the whole conversation is re-sent, not a summary of it. OpenRouter documents this as model fallbacks, and its `openrouter/free` router picks from the free pool automatically. Kilo Code does the same thing with `kilo-auto/free`.

02

The one thing almost everyone gets wrong

People paste a provider base URL straight into `ANTHROPIC_BASE_URL` and expect it to work. It cannot. Groq's endpoint answers at `/openai/v1/chat/completions` and expects an OpenAI request body. Claude

Code sends an Anthropic Messages body to `/v1/messages`. The two are different shapes, and pointing one at the other gives you a 404 or an HTTP 200 with a malformed response, which is a confusing error to debug because the connection itself is fine.

WATCH OUT

Anthropic states this in its own documentation: it does not endorse, maintain or audit third-party gateway products, and does not support routing Claude Code to non-Claude models through any gateway. The configuration works and nothing blocks you, but a session that breaks on a free model is yours to fix. Keep client work on Claude and send the bulk jobs to the free tier.

There is a second trap underneath that one. The moment you set a gateway credential, your `claude.ai` subscription stops being used. While **`ANTHROPIC_AUTH_TOKEN`** or an **`apiKeyHelper`** is active, requests carry that credential instead of your login, and the subscription's usage limits do not apply to them. Your login stays saved on the machine, unused, until you unset the variable. Remote Control and voice dictation also switch off, because both need a `claude.ai` identity.

03

The files

SOURCE CODE

The full provider list, with every base URL, model name, rate limit and data-usage footnote:

<https://github.com/mnfst/awesome-free-llm-apis>

~/.CLAUDE/SETTINGS.JSON

```
{
  "env": {
    "ANTHROPIC_BASE_URL": "https://your-gateway.example.com",
    "ANTHROPIC_AUTH_TOKEN": "sk-your-gateway-key",
    "CLAUDE_CODE_ENABLE_GATEWAY_MODEL_DISCOVERY": "1"
  }
}
```

Put it in `~/.claude/settings.json` and it applies to every project and to background agents. Put it in `.claude/settings.local.json` and it applies to one project only. Never put a credential in a project's `.claude/settings.json`, because that file gets committed. The third variable is optional: it makes Claude Code ask the gateway for its model catalogue at startup and add those names to the `/model` picker.

.ENV (ON THE GATEWAY, NEVER ON YOUR LAPTOP)

```
# Provider APIs (the companies that train the models)
GEMINI_API_KEY=          # aistudio.google.com/app/apikey
MISTRAL_API_KEY=         # console.mistral.ai/api-keys
COHERE_API_KEY=          # dashboard.cohere.com/api-keys
ZAI_API_KEY=              # open.bigmodel.cn/usercenter/apikeys

# Inference providers (third parties hosting open-weight models)
GROQ_API_KEY=             # console.groq.com/keys
NVIDIA_NIM_API_KEY=       # build.nvidia.com/explore/discover
OPENROUTER_API_KEY=       # openrouter.ai/keys
CLOUDFLARE_API_TOKEN=     # dash.cloudflare.com/profile/api-tokens
CLOUDFLARE_ACCOUNT_ID=    # needed in the Workers AI base URL path
HUGGINGFACE_TOKEN=        # huggingface.co/settings/tokens

# No key needed at all:
# OVHcloud https://oai.endpoints.kepler.ai.cloud.ovh.net/v1 2 RPM per IP
# LLM7.io https://api.llm7.io/v1 anonymous turbo models
# Kilo Code https://api.kilo.ai/api/gateway 200 req/hr per IP
```

BASE-URLS.TXT

```
Google Gemini https://generativelanguage.googleapis.com/v1beta
Mistral AI https://api.mistral.ai/v1
Cohere https://api.cohere.com/v2
Z AI (Zhipu) https://open.bigmodel.cn/api/paas/v4
Aion Labs https://api.aionlabs.ai/v1

Groq https://api.groq.com/openai/v1
NVIDIA NIM https://integrate.api.nvidia.com/v1
OpenRouter https://openrouter.ai/api/v1
Cloudflare https://api.cloudflare.com/client/v4/accounts/{account_id}/ai/run
Hugging Face https://router.huggingface.co/v1
Kilo Code https://api.kilo.ai/api/gateway
LLM7.io https://api.llm7.io/v1
ModelScope https://api-inference.modelscope.cn/v1
Ollama Cloud https://ollama.com/api (OpenAI-compatible at
https://ollama.com/v1)
OVHcloud https://oai.endpoints.kepler.ai.cloud.ovh.net/v1
SiliconFlow https://api.siliconflow.cn/v1
```

CLAUDE.MD (THE HANDOFF NOTE)

Model handoff

You may be a fallback model picked up mid-session after the previous model hit its rate limit. If so:

- The full conversation above is yours. Read it before answering.
- Do not restart, re-summarise, or re-introduce the task.
- State in one line which step you are resuming from, then continue.
- If a tool or capability you lack was being used, say so and propose the nearest thing you can do instead.

That last block is the part the video calls the handoff note. It is not a feature of the list and no gateway writes it for you. It is a section you add to your own **CLAUDE.md** or system prompt so an incoming fallback model knows it arrived mid-job and keeps going instead of starting over.

Set it up in Claude Code or Codex

1. Clone the list so you can read the footnotes offline: **git clone <https://github.com/mnfst/awesome-free-llm-apis>**. The README is the whole product. Read the footnote attached to any provider before you send it real work, because the data terms vary a lot between them.
2. Collect three keys to start. Google Gemini at aistudio.google.com/app/apikey, Groq at console.groq.com/keys, NVIDIA NIM at build.nvidia.com/explore/discover. Each takes about two minutes and none asks for a card.
3. Smoke-test each key against the provider directly, before any gateway exists. The two curl commands in the next section do this. If a raw key does not answer, no amount of gateway configuration will save it.
4. Stand up a gateway that exposes an Anthropic-format `/v1/messages` endpoint and holds those keys server-side. Configure its fallback chain in priority order, cheapest and fastest first. If you would rather not run one, OpenRouter's `openrouter/free` router gives you the fallback behaviour inside a single OpenAI-format endpoint, which suits Codex and your own scripts better than it suits Claude Code.
5. Ask Claude to add the keys to your shell profile rather than pasting them yourself: **add GROQ_API_KEY and NVIDIA_NIM_API_KEY to my ~/.zshrc, do not print the values back**. Then open a new terminal, because the current one will not see them.
6. Export the two Claude Code variables in that new terminal, run the verification curl in the next section, and only move them into `settings.json` once it returns JSON.
7. Start **claude** from the same shell and run **/status**. A successful first run shows an 'Anthropic base URL' line with your gateway address and an 'Auth token' line naming `ANTHROPIC_AUTH_TOKEN`. If the base URL line is missing entirely, the variable never reached the session.
8. On Codex, skip the gateway. Codex speaks the OpenAI format natively and every provider on this list is OpenAI SDK-compatible, so the base URL and key go straight into whatever provider setting your Codex version exposes. The translation layer only exists because of the Anthropic format.

QUICK TIPS

- NVIDIA NIM is the most generous entry on the list because its 10,000 requests per day is counted **per model**, not per account, across more than 100 models.
 - Cloudflare Workers AI gives 10,000 Neurons a day shared across all your Workers AI usage, resetting at 00:00 UTC. Going over fails the request rather than billing you, which makes it safe to leave in a chain.
 - OVHcloud AI Endpoints and LLM7.io both answer anonymous requests with no signup and no key, so they are the fastest way to prove your fallback logic works before you have registered anywhere.
-

The commands I actually run

VERIFY THE GATEWAY BEFORE OPENING CLAUDE CODE

```
curl -sS -w '\n%{http_code}\n' -X POST "$ANTHROPIC_BASE_URL/v1/messages" \
  -H "Authorization: Bearer $ANTHROPIC_AUTH_TOKEN" \
  -H "anthropic-version: 2023-06-01" \
  -H "content-type: application/json" \
  -d '{"model": "claude-sonnet-4-6", "max_tokens": 1, "messages": [{"role": "user",
"content": "."}]}'
```

SMOKE-TEST A RAW PROVIDER KEY

```
# Groq, no gateway involved, just proving the key works
curl -sS https://api.groq.com/openai/v1/chat/completions \
  -H "Authorization: Bearer $GROQ_API_KEY" \
  -H "content-type: application/json" \
  -d '{"model": "openai/gpt-oss-120b", "messages": [{"role": "user", "content": "say ok"}]}'

# OVHcloud, no key at all
curl -sS https://oai.endpoints.kepler.ai.cloud.ovh.net/v1/chat/completions \
  -H "content-type: application/json" \
  -d '{"model": "gpt-oss-20b", "messages": [{"role": "user", "content": "say ok"}]}'
```

HAVE CLAUDE BUILD THE FALLBACK CHAIN

Read the README at [PATH TO CLONED REPO] and build me a fallback chain config for my gateway.

Order: [PROVIDER 1] first, then [PROVIDER 2], then [PROVIDER 3].
 For each one use the exact base URL and model name from the README, not from memory.
 Flag any provider whose footnote says prompts may be used for training, and put those last.
 Output the config file only.

ROUTE THE CHEAP WORK OFF CLAUDE

For this task: [DESCRIBE THE TASK]

Decide whether it needs a frontier model or whether a free model handles it.
 If a free model handles it, name which one from [PROVIDER LIST] and why, then write the script that calls it. If it needs Claude, say so in one line and do the work here.

DIAGNOSE A ROUTED SESSION

My Claude Code session is routed through a gateway and failing with [PASTE THE EXACT ERROR].

Check in this order: whether /status shows the base URL line, whether the credential is in the header my gateway reads, and whether the gateway is enforcing a smaller context window than the model's native one. Tell me which of the three it is before suggesting a fix.

When it breaks

ERROR	FIX
401 naming an invalid or unrecognized token	The credential is in a header the gateway does not read. ANTHROPIC_AUTH_TOKEN sends it as Authorization: Bearer, ANTHROPIC_API_KEY sends it as x-api-key. Switch to the other variable and re-export.
API returned an empty or malformed response (HTTP 200)	The gateway answered with something that is not a Claude API response, usually an HTML error or a login page. Run the verification curl: it fails the same way and shows you the body.
Connection refused, or ENOTFOUND	Nothing answered at the base URL. The address is wrong, or a VPN or firewall is blocking the path. The curl test fails immediately with the same cause.
400 naming context_management or 'Extra inputs are not permitted'	The upstream rejects fields Claude Code sends to Anthropic-format endpoints. Set CLAUDE_CODE_DISABLE_EXPERIMENTAL_BETAS=1, which suppresses most pre-release fields.
ContextWindowExceededError, or 'prompt token count of N exceeds the limit of M'	The gateway enforces a smaller context than the model's native window and rewrote the error, so Claude Code did not compact on its own. Run /compact to recover, then set CLAUDE_CODE_AUTO_COMPACT_WINDOW to the gateway's limit.
Claude Code asks you to log in even though the curl test passed	A reachable base URL is not a credential. Set ANTHROPIC_AUTH_TOKEN somewhere Claude Code reads before first-run setup: a shell export or the env block in ~/.claude/settings.json.
A startup warning naming two credential sources	A gateway credential and a saved claude.ai login are both active. Unset the variable to use the login, or run /logout to use the gateway credential.
Remote Control or voice dictation unavailable	Expected. Both need a claude.ai identity, and Remote Control is disabled while ANTHROPIC_BASE_URL points at a non-Anthropic host. Unset the gateway variables to get them back.
Free provider returns 429 almost immediately	Check the rate limit column in the README for that exact model. OVHcloud's anonymous tier is 2 requests per minute per IP, and OpenRouter free models default to 50 requests per day each.

WHERE TO GRAB THE FREE KEYS

- Google AI Studio (Gemini)
<https://aistudio.google.com/app/apikey>
- Groq Console
<https://console.groq.com/keys>
- NVIDIA NIM
<https://build.nvidia.com/explore/discover>
- Mistral Console
<https://console.mistral.ai/api-keys>
- OpenRouter
<https://openrouter.ai/keys>
- Cloudflare Workers AI
<https://dash.cloudflare.com/profile/api-tokens>

Ready to go deeper?

This playbook is the short version. Inside the hub you get the full build walkthroughs, the prompt library, and the systems I use to ship this in a weekend.

Join the Build →

<https://hub.digicuratoragency.com/join>